Week 7 Lab: RTOS Communication — Sensor Pipeline

EE0827 Embedded System Applications

Overview

Build a multi-stage sensor data pipeline using FreeRTOS queues and mutexes. You will read ADC values, process them through a queue-based pipeline, and output results via UART with mutex protection.

Time allocation: 90 minutes

- Hardware & CubeMX setup: 20 min
- Queue and mutex implementation: 30 min
- Testing and debugging: 25 min
- Failure injection experiments: 15 min

Learning Objectives

1. Create and configure message queues in FreeRTOS
2. Implement the producer-consumer pattern with queues
3. Protect shared resources (UART) using mutexes
4. Debug queue overflow and mutex contention issues

Safety & Setup

> ⚠ Safety Checklist
>
> ☐ USB cable connected to Nucleo board (USB power only — bench supply OFF)
> ☐ Potentiometer wired correctly: **Pin 1 → 3.3V**, **Wiper → PA0**, **Pin 3 → GND**
> ☐ Serial terminal open at **115200 baud**

Hardware: Nucleo-F401RE, 10 kΩ potentiometer, breadboard, USB cable

Software: STM32CubeIDE with FreeRTOS, serial terminal (PuTTY / Tera Term)

Pre-Lab Questions

1. What is the minimum queue size if the producer sends 10 readings/s and the consumer batches every 4?

---

2. What happens if two tasks call `printf()` simultaneously without mutex protection?

---

3. Why can't we use a blocking mutex acquire inside an ISR?

Part 1

Hardware & CubeMX Setup

20 minutes

Step 1.1: Wire the Potentiometer

Potentiometer Pin	Connect To	Nucleo Header
Pin 1 (end)	3.3V	CN6 Pin 4
Pin 2 (wiper)	PA0	CN8 Pin 1 (A0)
Pin 3 (end)	GND	CN6 Pin 6

Verify with multimeter: rotate pot — voltage at PA0 should vary 0 V to 3.3 V.

Step 1.2: CubeMX Project

1. **New project** → Board: NUCLEO-F401RE → Name: `W07_Pipeline`
2. **ADC1 → IN0** — Resolution: 12 bits, all other defaults
3. **USART2** — Asynchronous, 115200 baud, 8N1
4. **Middleware → FREERTOS → CMSIS_V2**
    - `TOTAL_HEAP_SIZE`: 15360
    - `USE_NEWLIB_REENTRANT`: Enabled

Step 1.3: Create RTOS Objects

Queues tab:

Queue Name	Size	Item Size	Purpose
RawDataQueue	10	2 (`uint16_t`)	Raw ADC readings
ProcessedQueue	5	2 (`uint16_t`)	Averaged values

Mutexes tab:

Mutex Name	Type
UartMutex	Normal

Tasks tab:

Task Name	Priority	Stack	Entry Function
SensorTask	Normal	128	StartSensorTask
ProcessTask	Normal	128	StartProcessTask
DisplayTask	BelowNormal	256	StartDisplayTask

Why 256 words for DisplayTask?

`printf()` uses the calling task's stack for formatting. 128 words is too small when printf builds a formatted string — you will get a hard fault.

Generate code and open in CubeIDE.

Part 2

Implement the Pipeline

30 minutes

Step 2.1: Redirect printf to UART

In `freertos.c`, add inside the `USER CODE BEGIN 0` section:

```
/* USER CODE BEGIN 0 */
extern UART_HandleTypeDef huart2;

int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart2, (uint8_t*)ptr, len, HAL_MAX_DELAY);
    return len;
}
/* USER CODE END 0 */
```

Also add at the top: `#include <stdio.h>`

Step 2.2: SensorTask — Producer

```
void StartSensorTask(void *argument)
{
    uint16_t adc_value;

    for(;;)
    {
        HAL_ADC_Start(&hadc1);
        if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) {
```

```
            adc_value = HAL_ADC_GetValue(&hadc1);

            osStatus_t status = osMessageQueuePut(
                RawDataQueueHandle, &adc_value, 0, 10);

            if (status != osOK) {
                // Queue full - data lost
            }
        }
        HAL_ADC_Stop(&hadc1);
        osDelay(100);  // 10 Hz sample rate
    }
}
```

Step 2.3: ProcessTask — Filter

```
void StartProcessTask(void *argument)
{
    uint16_t raw_value, average;
    uint32_t sum = 0;
    uint8_t count = 0;

    for(;;)
    {
        osStatus_t status = osMessageQueueGet(
            RawDataQueueHandle, &raw_value, NULL, osWaitForever);

        if (status == osOK) {
            sum += raw_value;
            if (++count >= 4) {
                average = sum / 4;
                osMessageQueuePut(
                    ProcessedQueueHandle, &average, 0, 10);
                sum = 0;
                count = 0;
            }
        }
    }
}
```

Step 2.4: DisplayTask — Consumer

```c
void StartDisplayTask(void *argument)
{
    uint16_t avg_value;
    uint32_t display_count = 0;
    float voltage;

    for(;;)
    {
        osStatus_t status = osMessageQueueGet(
            ProcessedQueueHandle, &avg_value, NULL, osWaitForever);

        if (status == osOK) {
            display_count++;
            voltage = (avg_value / 4095.0f) * 3.3f;

            osMutexAcquire(UartMutexHandle, osWaitForever);
            printf("[%04lu] ADC Avg: %4u  Voltage: %.2fV\r\n",
                   display_count, avg_value, voltage);
            osMutexRelease(UartMutexHandle);
        }
    }
}
```

Step 2.5: Build and Flash

1. **Build** (Ctrl+B) — fix any errors
2. **Flash** to board (Run or F11)
3. **Open terminal** at 115200 baud
4. **Rotate potentiometer** — observe values changing

Expected output:

```
[0001] ADC Avg: 2048  Voltage: 1.65V
[0002] ADC Avg: 2050  Voltage: 1.65V
[0003] ADC Avg: 3000  Voltage: 2.42V
```

Checkpoint

You should see continuous output updating at ~2.5 Hz (one line per 4 ADC samples). Values should change smoothly as you turn the pot. If you see nothing, check the **Common Issues** table on the last page.

Part 3

Testing & Debugging

25 minutes

Step 3.1: Monitor Queue Depth

Add periodic debug output to SensorTask (inside the `for` loop, after the queue put):

```
static uint32_t dbg = 0;
if (++dbg >= 50) {  // Every 5 seconds
    uint32_t n = osMessageQueueGetCount(RawDataQueueHandle);
    osMutexAcquire(UartMutexHandle, osWaitForever);
    printf("[DEBUG] RawQueue items: %lu\r\n", n);
    osMutexRelease(UartMutexHandle);
    dbg = 0;
}
```

Record typical queue depth: _____ items (expected: 0–4)

Step 3.2: Demonstrate Mutex Problem

Temporarily comment out mutex acquire/release in DisplayTask, and add an unprotected print in SensorTask:

```
// In SensorTask loop (unprotected):
printf("S");
```

Observe output — you should see garbled/interleaved text:

Without Mutex	With Mutex
`[00S01] ADCS Avg: 20S48`	`[0001] ADC Avg: 2048`

Restore mutex protection before continuing!

Comment out the unprotected `printf("S")` and uncomment the mutex acquire/release lines.

Demonstrated garbled output? Yes

Step 3.3: Queue Overflow Test

In CubeMX, **temporarily** change `RawDataQueue` size from 10 to **2**. Regenerate and rebuild.

Add overflow counting in SensorTask:

```
if (status == osErrorTimeout) {
    overflow_count++;  // declare as static uint32_t
}
```

Observed overflows? Yes — **Restore queue size to 10** when done.

Part 4

Failure Injection

15 minutes

For each test: **predict**, **test**, **restore**.

Test 1: Remove SensorTask delay

Comment out `osDelay(100);`. **Predict**, then test. **Restore when done.**

Test 2: Stall the consumer

Add `osDelay(10000);` at top of ProcessTask loop. **Predict**, then test. **Restore when done.**

Test 3: Deadlock Analysis (pen-and-paper)

```
// Task A                          // Task B
osMutexAcquire(X, osWaitForever); osMutexAcquire(Y, osWaitForever);
osMutexAcquire(Y, osWaitForever); osMutexAcquire(X, osWaitForever);
osMutexRelease(Y);                osMutexRelease(X);
osMutexRelease(X);                osMutexRelease(Y);
```

Deadlock? Yes No — **Why?** _____

Measurement Summary

ID	Measurement	Expected	Measured	Pass?
M1	ADC sample rate	10 Hz		
M2	Averaged output rate	2.5 Hz		
M3	RawQueue typical depth	0–4 items		
M4	Queue overflow (normal)	0		
M5	UART output (with mutex)	Clean		
M6	Voltage at pot full CCW	~0 V		
M7	Voltage at pot mid	~1.65 V		
M8	Voltage at pot full CW	~3.3 V		

Analysis Questions

1. Why does ProcessTask use `osWaitForever` while SensorTask uses a 10 ms timeout?

_____

2. What would happen if you used `HAL_Delay()` instead of `osDelay()` in SensorTask?

_____

3. How would you modify this design to handle 100 Hz sampling?

_____

Deliverables

Completion Checklist

- ☐ Working 3-task pipeline (Sensor → Process → Display)
- ☐ Queue-based data transfer demonstrated
- ☐ Mutex protection demonstrated (showed garbled vs clean)
- ☐ Queue overflow test performed
- ☐ Measurement table completed
- ☐ Analysis questions answered

This lab is **ungraded practice**.

Common Issues

Issue	Fix
No UART output	Add `_write()` redirect, check baud = 115200
ADC reads 0	Set PA0 to Analog in CubeMX, enable ADC clock
Hard fault on printf	Increase DisplayTask stack to 256 words
Queue data wrong	Item size must be 2 (`uint16_t`), not `sizeof(struct)`
Tasks don't start	Verify `osKernelStart()` is called in `main.c`

Key Takeaways

1. **Queues decouple tasks** — producer and consumer run independently
2. **Mutexes prevent interleaving** — essential for shared peripherals like UART
3. **Always use timeouts** — prevents permanent deadlocks
4. **Pipeline pattern** scales to multi-stage data processing